

Odds Ratio - Robo-Advisory Agéntico

Track 3: Robo-Advisory y Automatización de Estrategias de Inversión Hackathon Agentic Scale · Ecuador Tech Week 2026 · Equipo Odds Ratio (Dataclub, ESPOL)

Proyecto demostrativo con **datos ficticios y simulaciones ilustrativas**. El sistema **nunca ejecuta órdenes reales ni promete rentabilidad**: toda acción sensible queda como propuesta sujeta a aprobación de un asesor humano.

¿Qué hace?

Un robo-advisor con arquitectura agéntica (LangGraph) que:

1. **Perfila al inversionista** con reglas transparentes y versionadas (YAML v1.1.0, 10 preguntas + 1 opcional) - cada respuesta explica su influencia.
2. **Actualiza un posterior bayesiano** (Beta-Binomial) del perfil con cada decisión del asesor: el sistema aprende de sus supervisores humanos.
3. **Genera una propuesta de portafolio** desde un catálogo ficticio + **fan chart** de proyección (modelo Normal-Normal, percentiles 5/25/50/75/95 - nunca una línea determinística), anclado a priors bancarios reales.
4. **Valida la asignación con Markowitz** (Teorema de Separación de Dos Fondos) como segunda opinión cuantitativa, sin reemplazar la asignación oficial por reglas.
5. **Explica todo con Google Gemini**, bajo un **guardrail antialucinación activo**: si el LLM inventa una cifra, su salida se descarta automáticamente.
6. **Se detiene en un checkpoint humano** (interrupt de LangGraph): un asesor autorizado aprueba/edita/rechaza, y su decisión queda en una **auditoría append-only** con versiones de reglas y modelos.
7. **Chat IA de atención al cliente** (ORAI): consultas generales 24/7 con guardrails duros - nunca da cifras (se filtran automáticamente) y escala a asesor humano cuando detecta solicitudes de inversión.
8. **Panel Operativo con login** (solo funcionarios): revisión de propuestas, las 3 Historias por cliente, ranking bayesiano de clientes (Thompson sampling), reporte de calidad del cuestionario, Modo Autopiloto, Modo Piloto y auditoría.
9. **MODO AUTOPILOTO (autonomía por excepción)**: el sistema **se gana la autonomía estadísticamente**. Cuando la confianza bayesiana de un cliente supera el umbral configurado y no hay alertas de Cumplimiento, el Autopiloto aprueba solo - firmando la auditoría como ODDS-Autopilot v1. Apagado por defecto; nunca ejecuta órdenes reales.
10. **Muestreo estratificado + intencional**: cada cliente se etiqueta con Estrato A (edad), B (capacidad económica) y C (experiencia); Modo Piloto captura en silencio tiempo/vacilación por pregunta y feedback puntual para detectar preguntas confusas (z-score) - todo consolidado en un reporte de calidad con Alfa de Cronbach real.

Credenciales demo del Panel Operativo

Usuario	Clave	Configurable vía
operador	oddsratio2026	env OPERATOR_USER / OPERATOR_PASS

Stack

Capa	Tecnología
Orquestación agéntica	LangGraph (StateGraph + checkpoint SQLite + interrupt humano)
LLM	Google Gemini API (gemini-2.5-flash) vía interfaz intercambiable LLMPProvider
Backend	Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy + SQLite, PyYAML, NumPy
Estadística	Beta-Binomial + Normal-Normal conjugado + Markowitz (media-varianza) + Alfa de Cronbach
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS, Recharts
Despliegue	Docker Compose (backend + frontend + volumen SQLite)

Cómo correr

Opción A - Docker (recomendada para la VM en la nube)

```
# 1. (Opcional) exporta tu API key de Gemini - sin ella usa el proveedor mock
export GEMINI_API_KEY=tu_api_key
# 2. En despliegue remoto, apunta el navegador al backend público:
export NEXT_PUBLIC_API_URL=http://TU_IP_O_DOMINIO:8000

docker compose up -d --build
# Frontend: http://localhost:3000 · API: http://localhost:8000/docs
```

Opción B - Local sin Docker

```
# ---- Backend (puerto 8000) ----
cd backend
pip install -r requirements.txt
cp ../env.example .env # pon tu GEMINI_API_KEY (opcional)
uvicorn app.main:app --port 8000

# ---- Frontend (puerto 3000) ----
cd frontend
npm install
npm run dev
```

Flujo de la demo (3 pasos)

- http://localhost:3000/onboarding** - responde el cuestionario -> el grafo genera la propuesta y se interrumpe.
- /propuesta?id=N** - perfil, confianza bayesiana, distribución, fan chart, comparación con Markowitz y explicación de Gemini (con disclaimers visibles).
- /asesor** - aprueba/edita/rechaza -> el grafo se reanuda, registra auditoría y actualiza el posterior (verás α , β y la nueva confianza en pantalla).

Tests

```
cd backend
```

Archivo	Qué prueba
tests/test_rules_engine.py	Perfilamiento: umbrales, influencias, versiones, 10 preguntas obligatorias, validación de entradas.
tests/test_bayes_posterior.py	Actualización Beta-Binomial por decisión, intervalos de credibilidad, reproducibilidad.
tests/test_portfolio_optimizer.py	Markowitz: pesos no negativos, suma 100%, aversión al riesgo por perfil.
tests/test_graph_nodes.py	Nodos LangGraph con LLM mockeado : guardrail antialucinación, flujo E2E interrupt->resume, doble decisión bloqueada.
tests/test_chat_ranking_auth.py	Chat con guardrail de cifras y escalamiento a humano, login/token de operadores, ranking bayesiano ordenado y autoalimentado, priors bancarios cargados.
tests/test_autopilot.py	Modo Autopiloto: apagado por defecto, confianza bajo umbral -> humano, autonomía ganada, alerta de Cumplimiento revoca la autonomía.
tests/test_muestreo.py	Estratos A/B/C, Modo Piloto, Alfa de Cronbach real, casos críticos por z-score.
tests/test_review_history.py	Las 3 Historias consolidadas para el asesor.

Estructura

```

odds-ratio/
+-- backend/
|   +-- app/
|   |   +-- main.py           # FastAPI (solo capa API)
|   |   +-- graph/           # grafo LangGraph: state, nodos, build
|   |   +-- bayes/           # Beta-Binomial + Normal-Normal + Markowitz + ranking
|   |   +-- llm/              # LLMProvider: Gemini / mock intercambiables
|   |   +-- rules/            # risk_rules_v1.yaml + motor determinístico + estratos
|   |   +-- analytics/        # reporte de calidad (Cronbach, z-score)
|   |   +-- data/             # DataProvider ficticio (catálogo + retornos + priors bancarios)
|   |   +-- workflows/        # Asesoría, Cumplimiento, Autopiloto, Modo Piloto
|   |   +-- routers/          # profiling · proposals · forecast · review · chat · clients
|   +-- tests/                # 65 tests (pytest)
+-- frontend/                 # Next.js 14: landing · onboarding · propuesta · asesor · chat
+-- docker-compose.yml
+-- DOCUMENTO_EXPLICATIVO.md # entregable oficial (arquitectura + estadística + track)
+-- README.md

```

Mitigación de riesgos / antialucinación

- Todo número sale de **reglas determinísticas o modelos estadísticos versionados** - el LLM jamás calcula.
- El nodo explicacion_llm recibe un **JSON de solo lectura**; su salida pasa por validar_consistencia_numerica() y se **descarta si contiene cifras que no estén en la entrada** (queda registrado en auditoría).
- Disclaimers visibles en UI, API y textos generados.

- Auditoría **append-only**: decisión, asesor, snapshot completo, versión de reglas y del modelo bayesiano.